为什么项目不使用 LangChain / LangGraph？

RAGKit 与 AgentRun 技术选型面试问答

本资料围绕五个高频追问，帮助你把“为什么没用框架”回答成一次有依据的技术选型说明。

> 核心表达：不是不会使用框架，而是根据项目目标决定哪些能力自己实现、哪些通用能力优先复用。

适用项目：RAGKit（中文技术文档 RAG）｜AgentRun（可靠 Agent 执行引擎）

一、先把两个框架的定位说清楚

LangChain：提供 LLM
应用开发组件与抽象，常见内容包括模型调用、Prompt、文档加载、Retriever、工具集成等。

LangGraph：提供有状态 Agent 工作流编排能力，常见内容包括
State、节点、条件分支、循环、Checkpoint、中断与恢复等。

> 面试时不要把它们说成“做不到”或“没有价值”。更准确的表达是：框架能解决很多通用问题，但项目的核心研究目标决定了我选择自主实现哪些部分。

两个项目的选型理由

项目	核心目标	自主实现的重点
RAGKit	理解并验证 RAG 检索链路	文本处理、Chunk、BM25、Dense、融合、Rerank、评测
AgentRun	研究 Agent 的可靠执行	事件溯源、幂等、租约、Epoch Fencing、对账与人工复核

二、五个高频面试问题

1. LangChain 有什么缺点？

推荐回答

> 我认为 LangChain 的问题不是功能不足，而是它提供了比较多的抽象层。
> 第一，抽象层比较多。对于简单场景，可能引入不必要的复杂度，增加学习和调试成本。
> 第二，框架封装了不少底层实现。如果不了解内部机制，遇到问题时可能不容易定位。
> 第三，框架 API、组件生态和依赖版本会持续演进，需要考虑兼容性与维护成本。
> 第四，高度定制化的业务场景，通用抽象不一定完全匹配，可能需要扩展或绕过框架。
> 当然，LangChain 的优势是快速集成模型、工具和检索组件。关键是根据项目需求选择合适的抽象层。

结合 RAGKit 的追问：为什么不直接使用 Retriever？

> 因为 RAGKit 的目标是研究检索链路。我希望自己实现 BM25、Dense Retrieval、多路召回、RRF 融合和 Rerank，理解各模块的输入输出与效果。如果直接使用现成 Retriever，开发会更快，但不利于我深入研究底层检索机制。

回答边界：不要笼统地说 LangChain 性能差、很臃肿或不适合生产。要说明具体场景，以及抽象带来的收益和成本。

2. LangGraph 不是也支持持久化和恢复吗？

推荐回答

> 是的，LangGraph 支持通过 Checkpoint 持久化图的状态，并在中断后恢复执行。
> 但我会区分工作流状态恢复和外部副作用的一致性。Checkpoint 可以保存当前节点、State 等工作流信息；但如果 Agent 调用了转账工具，外部转账已经成功，而程序在收到返回结果之前崩溃，恢复工作流状态本身并不能直接证明转账是否成功。
> 这时仍然需要稳定的幂等键、外部状态查询和对账机制，确认操作是否已经生效。
> 所以 AgentRun 关注的不只是工作流持久化，而是外部副作用发生后，如何处理执行结果不确定的问题。

崩溃场景示意

步骤	发生的事情	系统需要做什么
1	Agent 调用转账工具	使用稳定 request_id / step_key
2	外部账本转账成功	外部副作用已经发生
3	程序收到结果前崩溃	本地日志可能没有成功结果
4	程序恢复执行	不能盲目再次转账
5	Reconcile 查询外部账本	确认已执行、未执行或进入人工复核

> 关键句：Checkpoint 不等于跨系统事务，也不能单独保证外部操作恰好执行一次。

重要边界：不要说 LangGraph 不支持恢复外部操作。LangGraph
可以结合任务封装、重试、幂等和外部查询来处理部分问题；准确说法是 Checkpoint 本身不能自动解决跨系统副作用的一致性。

3. 如果让你重新做 RAGKit，会使用 LangChain 吗？

推荐回答

> 我会根据项目目标区分。如果是为了快速开发一个应用级 RAG 系统，我会考虑使用 LangChain，因为它提供了模型接口、文档加载器、Retriever 和向量数据库集成，可以减少重复开发。
> 但如果项目重点仍然是研究检索算法，我会保留核心检索模块的自主实现。例如 BM25、Dense Retrieval、RRF 融合、Rerank 和 Recall@K 评测，我仍然倾向于自己实现，或者至少掌握其底层逻辑。
> 所以我不会完全排斥框架，而是把成熟框架用于通用基础设施，把需要研究和定制的核心模块自己实现。

面试官考察点

- 是否知道框架能提高开发效率，而不是为了“纯手写”而拒绝复用。
- 是否能区分通用组件与项目核心算法。
- 是否能根据项目目标、时间成本和维护成本调整技术选型。

4. LangGraph 能不能实现你的 AgentRun？

推荐回答

> 我认为可以实现其中很多功能。LangGraph 支持 State 管理、节点调度、条件分支、Checkpoint 和中断恢复。如果只从功能实现角度看，可以基于 LangGraph 构建类似的 Agent 执行流程。
> 但 AgentRun 的设计目标不是实现一个普通的 Agent 工作流，而是研究可靠执行机制。租约、Epoch Fencing、稳定幂等键、事件溯源以及 Reconcile 对账，主要解决多进程竞争、外部副作用和崩溃后的状态不确定性问题。
> 所以我选择自己实现执行引擎，是为了深入理解这些机制。未来也可以考虑让 LangGraph 负责上层 Agent 编排，而 AgentRun 负责底层可靠执行。

一种可行的分层思路

层次	职责	示例
上层：LangGraph	Agent 工作流编排	State、节点、条件分支、循环
底层：AgentRun	可靠执行	EventLog、Lease、Fencing、幂等、Reconcile
外部系统	真实副作用	数据库、账本、API、业务工具

注意：这是未来可采用的分层设计思路，不代表当前 AgentRun 已经实现 LangGraph 集成。

5. 自己造轮子是不是没有必要？

推荐回答

> 我认为不能一概而论，关键是看自己实现的东西是否具有研究价值或业务价值。
> 如果是成熟、通用、没有必要定制的基础组件，例如 HTTP 客户端、数据库驱动，我会优先使用成熟的第三方库。
> 但如果是项目的核心机制，例如 RAGKit 中的检索融合算法，或者 AgentRun 中的事件溯源、租约和崩溃恢复机制，我认为自己实现是有价值的，因为这样可以深入理解底层原理，也能按需求定制。
> 当然，自主实现会增加开发和维护成本，所以我不会为了造轮子而造轮子，而是根据技术价值、开发成本和长期维护成本进行权衡。

三、五题速记表

面试问题	一句话核心
LangChain 有什么缺点？	抽象提高便利性，也可能增加复杂度和调试成本。
LangGraph 支持恢复吗？	支持，但工作流状态恢复不等于外部副作用一致性。
RAGKit 重做会用吗？	通用组件可以用，核心检索算法仍可自主实现。
LangGraph 能实现 AgentRun 吗？	很多功能可以，但 AgentRun 关注可靠执行问题。
造轮子有必要吗？	核心机制值得研究，成熟通用组件优先复用。

四、回答时的三个注意事项

- 不要贬低框架：不要说 LangChain / LangGraph 不行、不能用于生产，或者无法实现你的功能。
- 不要夸大自研：不要暗示自己实现的机制天然比成熟框架更可靠。要能说明测试、边界和代价。
- 不要只说为了学习：把学习动机进一步落到具体技术目标，例如检索算法可控性、外部副作用一致性、故障恢复和可观测性。

建议的总括表达

> 我的选型原则是：通用、成熟、非核心的能力优先复用；与项目核心目标直接相关、需要深入理解或高度定制的机制，才考虑自主实现。RAGKit 主要是为了掌握检索链路，AgentRun 主要是为了研究可靠执行。

五、模拟面试练习

练习规则：每次只回答一个问题；先用 30～60 秒讲清楚结论，再用项目中的具体设计举例。回答后主动检查自己是否说明了“为什么、解决什么问题、代价是什么”。

- 第一轮：不看资料，连续回答上面的五个问题。
- 第二轮：重点练习 Checkpoint 与外部副作用的区别。
- 第三轮：让面试官追问“为什么不用成熟框架”“你自己的实现有什么边界”。

> 最终目标：不是背出一段标准答案，而是能围绕项目中的真实设计，解释技术选择、替代方案、收益与成本。